Как проверить наше решение на LinQ TPU

Доступ к стенду, подготовка моделей, подключение адаптера, проверка сайта и воспроизводимые замеры.

Сначала важное

По документам предоставляется **удалённый сервис инференса LinQ TPU**. Это не подтверждённая CUDA-машина для обучения. PyTorch/CUDA-команды и nvidia-smi не доказывают работу на LinQ.

Уже есть	Что нужно выполнить на стенде
ONNX-модели, тестовые примеры и интерфейс backend	Прочитать SDK, скомпилировать TPU-программы и реализовать реальные API-вызовы
Локальный сайт, LIVE, клипы, журнал и правила	Запустить продукт через адаптер и подтвердить оборудование логами сервиса
Локальная оценка падений: event P/R/F1 = 0,90	Проверить изменение качества после переноса; >0,95 сейчас не подтверждено

Главное ограничение: на дату подготовки инструкции конкретный адаптер LinQ ещё не реализован, SDK не удалось прочитать по внешним ссылкам. Поэтому шаги доступа можно пройти самостоятельно; этап интеграции требует разработки по реальному примеру SDK. В документе нет выдуманных команд компилятора.

Маршрут проверки: локальная подготовка → VPN и штатный ResNet50 → компиляция наших моделей → адаптер → замеры → сайт → комплект доказательств.

ПРАКТИЧЕСКАЯ ИНСТРУКЦИЯ / ШАГ 01

Подготовить проект до VPN

Работайте в PowerShell из папки проекта. До переключения VPN сохраните эту инструкцию, исходники SDK и нужные зависимости локально: во время подключения связь с чатом может прерываться.

```
cd C:\Users\User\Desktop\mosprom2026
git status --short
git lfs install
git lfs pull
git lfs fsck
python -m pytest -q
node --test tests/test_camera_preview.cjs
```

Git LFS нужен для настоящих весов: обычный clone может оставить короткие текстовые указатели вместо моделей. При новом развёртывании сначала установите зависимости из requirements.txt и совместимую пару torch/torchvision для локального ПК. На TPU-сервер не переносите автоматически Windows-окружение.

Проверить состав и зафиксировать версию

```
git rev-parse HEAD
Get-FileHash data/models/yolov8n-pose.onnx -Algorithm SHA256
Get-FileHash data/models/fall-resnet18.onnx -Algorithm SHA256
New-Item -ItemType Directory -Force work/linq-check
```

Что подготовить заранее

Модели из data/models; data/linq-calibration; короткие разрешённые видео; docs/quality_report.json; docs/evaluation_protocol.json; docs/body_onnx_smoke.json; docs/onnx_smoke.json; data/models/scenario-manifest.json. Полный train/test URFD не включён в репозиторий: для оценки его нужно подготовить отдельно.

Отдельно сохранить SDK и пример API

Ссылки из инструкции организаторов: Framework / SDK и Python API example. В первом пакете нужен tpu_framework.pdf, во втором - usage_example_python_api.zip. Если ссылки недоступны, запросите файлы у организаторов; без них перенос пока не завершить.

ПРАКТИЧЕСКАЯ ИНСТРУКЦИЯ / ШАГ 02

Подключиться к стенду

1. VPN

Откройте OpenVPN Connect и импортированный профиль LinQ-Hackathon. При конфликте отключите другой VPN, затем нажмите Connect. Введите выданные капитану **VPN** логин и пароль. Пароли не добавляйте в ноутбук, код, скриншоты и git.

```
Resolve-DnsName hackathon.lin-q.ru
Test-NetConnection hackathon.lin-q.ru -Port 80
route print
```

Успех: клиент показывает Connected, имя разрешается, TCP-проверка порта 80 проходит. Ping может быть запрещён; его отсутствие само по себе не означает неисправность стенда. Если сайт недоступен, приложите организаторам время ошибки, DNS/TCP-результаты и очищенный от секретов лог OpenVPN.

2. Jupyter

Откройте Jupyter организаторов и войдите отдельной учётной записью **Jupyter**. Регистр логина важен. В рабочем пространстве должны быть examples и packages. Не переносите настройки из чужой команды.

3. Сначала штатный пример

В examples найдите пример ResNet50, прочитайте его вводные ячейки и выполните в указанной последовательности. Замените демонстрационный логин на логин команды и используйте свою загруженную TPU-программу. Сохраните вывод, время запуска, имя программы и идентификатор выполнения, если сервис его выдаёт.

Критерий прохождения

Получен осмысленный выход штатной модели через API LinQ и подтверждено выполнение на TPU. Просто открытый Jupyter, установленный пакет или успешный HTTP-запрос этого не подтверждают. ResNet50 проверяет доступ, но ещё не переносит наш продукт.

В ноутбуке допустимо выполнить import sys; print(sys.version), чтобы узнать Python. Не устанавливайте пакеты поверх окружения организаторов без проверки требований примера; пользуйтесь разрешённым отдельным окружением, если оно предусмотрено.

ПРАКТИЧЕСКАЯ ИНСТРУКЦИЯ / ШАГ 03

Скомпилировать наши модели

Программа TPU содержит граф и веса в формате LinQ. **Файл .onnx не является готовой TPU-программой.** Организаторы указывают: квантование и компиляция выполняются локально, затем программа загружается в сервис.

Модель	Контракт и приоритет
yolov8n-pose.onnx	Вход FP32 NCHW 1×3×640×640; выход 1×56×8400; opset 12. Детектор людей и 17 COCO-точек; decode/NMS на хосте.
fall-resnet18.onnx	Вход FP32 NCHW 1×3×224×224; выход 1×2 логита; opset 12. Проверка переноса RGB-классификатора падений.
Дополнительные ONNX	PPE13, fire/smoke, smoking, construction/machinery, COCO nano и COCO small. Точные имена, классы и хеши: scenario-manifest.json.

Порядок работы с SDK

Прочитайте разделы установки и end-to-end примера в tpu_framework.pdf. Зафиксируйте поддержанные ОС, Python/ONNX, версию SDK и список операций. Установите компилятор в рекомендованной среде; Windows-совместимость заранее не подтверждена. Сначала воспроизведите компиляцию примера автора.

Затем проверьте импорт нашего статического ONNX, выполните документированные калибровку/квантование, сборку TPU-программы и её валидацию. Сохраните stdout/stderr, параметры точности, список неподдержанных узлов и хеш полученного файла. Команду берите из **вашей версии SDK**, а не из примеров для других ускорителей.

Если pose-граф не поддержан

Проверьте возможность оставить decode и NMS на CPU. При необходимости используйте совместимый детектор людей для зон/движения. Любая замена модели требует новой оценки. Комбинация CPU-позы и TPU-детектора должна быть явно указана; прежние метрики нельзя автоматически приписать ей.

ПРАКТИЧЕСКАЯ ИНСТРУКЦИЯ / ШАГ 04

Калибровка и совпадение тензоров

Кадры для INT8 берём только из train

В data/linq-calibration есть 6 обучающих входов ResNet18 в .npz с input и expected_logits. Они подходят для короткой проверки формата, но недостаточны для репрезентативной калибровки. Подготовьте разнообразные разрешённые train-кадры: люди в СИЗ и без них, дальние планы, разные освещение и позы, нормальные действия. Test не используется для настройки.

Часть	Что должно совпадать
Pose / объекты	Letterbox с сохранением пропорций до размера экспорта; BGR → RGB; нормализация, layout и dtype как у исходной модели. Для INT8 учесть scale, zero-point и signed/unsigned SDK.
RGB ResNet18	BGR → RGB; PIL bilinear resize 224×224; /255; mean [0.485,0.456,0.406], std [0.229,0.224,0.225]; NCHW. Применить softmax к двум логитам, взять probability[1].
Координаты	После decode/NMS вернуть bbox и keypoints в пиксели исходного кадра с учётом padding и масштаба letterbox.

Проверка на одинаковых кадрах

Передайте идентичный вход в ONNX Runtime и TPU-программу. Сохраните shape/dtype входа и выходов, logits, вероятности, bbox, keypoints, max/mean absolute error. Проверьте отсутствие NaN/Inf, порядок классов и направление координат на нескольких реальных кадрах.

Ожидания к сравнению

Для FP32-экспорта уже есть локальные smoke-отчёты в docs/body_onnx_smoke.json и docs/onnx_smoke.json. Ошибки порядка 1e-6 у ResNet18 относятся к PyTorch ↔ ONNX, а не к INT8 TPU. Допуск для квантованного исполнения выбирается по влиянию на детекции и итоговые события, не копируется из FP32-теста.

SDK может требовать другой layout или целочисленный буфер. Не меняйте preprocessing «на глаз»: сверяйтесь с входным контрактом скомпилированной программы. Деквантование выхода выполняйте один раз.

ПРАКТИЧЕСКАЯ ИНСТРУКЦИЯ / ШАГ 05

Подключить адаптер к продукту

Точка интеграции - app/inference.py. Отдельный модуль, например my_linq.py в корне проекта, экспортирует create_backend(confidence=...). Никаких реальных API-вызовов LinQ в проекте пока нет: их нужно взять из рабочего примера организаторов.

```
class LinQBackend:
    name = "LinQ TPU / real program name"

    def predict(self, bgr_frame):
        # preprocess -> actual SDK/API -> receive output
        # dequantize -> decode -> NMS -> original pixels
        # return list[app.core.Person]
        ...

def create_backend(confidence=0.4):
    return LinQBackend()
```

Каждый Person содержит box=(x1,y1,x2,y2), confidence и, при поддержке, keypoints размером 17×3. track_id присваивается общим трекером. Низкоуверенные детекции до порога 0,1 сохраняются для второго прохода ByteTrack; итоговый порог берётся из конфигурации.

Падения и дополнительные сценарии

Для RGB-падений после predict устанавливайте scene_state={"lying_probability": p, "scope": "scene"}. Если этого нет, старые RGB-метрики к backend не относятся. Для дополнительных сценариев нужны configure_scenarios(config) и objects_state: list[Observation] из app/scenario_models.py, с bbox в исходных пикселях, label, confidence и model. Изучите структуру Observation в коде.

```
$env:LINQ_ADAPTER = "my_linq:create_backend"
python -m uvicorn app.server:app --host 127.0.0.1 --port 8765
```

Проверка подмены backend

Переменная задаётся в том же терминале до запуска сервера. Остановите прежний процесс на 8765, иначе увидите локальную версию. Ошибочный адаптер должен завершить прогон ошибкой. Успешная обработка через PyTorch не засчитывается как запуск на TPU.

ПРАКТИЧЕСКАЯ ИНСТРУКЦИЯ / ШАГ 06

Замерить скорость воспроизводимо

В проект добавлен scripts/benchmark_backend.py. Он вызывает реальный backend без веб-сервера, сохраняет config, хеш видео, покадровые выходы и задержки. Для сравнения используйте одинаковое видео, stride, число кадров и набор сценариев; каждый результат пишите в новый файл.

Локальный эталон до VPN / до переключения backend

```
Remove-Item Env:LINQ_ADAPTER -ErrorAction SilentlyContinue
python scripts/benchmark_backend.py `
  --video data/datasets/urfd/fall-01-cam0-rgb.mp4 `
  --warmup 10 --frames 100 `
  --output work/linq-check/local-fall.json
```

Измерение LinQ после реализации адаптера

```
$env:LINQ_ADAPTER = "my_linq:create_backend"
python scripts/benchmark_backend.py `
  --require-adapter `
  --video data/datasets/urfd/fall-01-cam0-rgb.mp4 `
  --warmup 10 --frames 100 `
  --output work/linq-check/linq-fall.json
```

Если видео содержит меньше 100 выбранных кадров, отчёт укажет фактически обработанное число. Для дополнительного набора сценариев сохраните RunConfig в JSON и добавьте --config путь/к/config.json. Fall-бенчмарк требует scene_state; поза без RGB-состояния не принимается за эквивалентный результат.

Что именно измеряется

predict_ms: p50/p95/mean для синхронного predict, включая preprocessing, передачу данных/API и postprocessing. predict_throughput_fps - пропускная способность этих вызовов, без декодирования, трекера, правил и сохранения клипов. Это **не время ядра TPU и не FPS полного продукта**. Для времени ядра используйте только данные сервиса/SDK.

Отдельно повторите полный прогон через сайт: 1 поток и 2 одновременных потока, реальные разрешения и включённые модели. Телефон использует дополнительные crop-проходы и заметно меняет стоимость кадра. Сохраните статистику прогонов и отметьте сетевую задержку.

ПРАКТИЧЕСКАЯ ИНСТРУКЦИЯ / ШАГ 07

Проверить сайт и события

Доступ и предприятие

Для версии с ролями при первом запуске создайте администратора платформы командой python scripts/create_admin.py из папки проекта. Введите собственный пароль скрыто, войдите на http://127.0.0.1:8765/ и откройте нужное предприятие. Новый клиент подаёт заявку; до одобрения его камеры недоступны. Кнопки анализа и управления источниками доступны локальному админу и специалисту по охране труда.

Режим «Анализ записей»

Загрузите короткое разрешённое видео. Назовите камеру, включите нужные сценарии, получите кадр и нарисуйте зоны. Для страховочной привязи обязательна зона «Работа на высоте». Запустите анализ и дождитесь завершения. Проверьте bbox, тип события, начало и время тревоги, отсутствие дублей, кадр-доказательство и воспроизводимый MP4-клип.

Откройте отчёт JSON записи и проверьте stats.backend. В health поле backend показывает конфигурацию адаптера, а не доказательство аппаратного выполнения. Поле linq_verified сейчас остаётся false автоматически: подтверждение оформляйте отдельным отчётом и логами SDK.

Режим «Прямой эфир»

Подключите веб-камеру/USB или разрешённый RTSP/HTTP-источник. Получите кадр, сохраните зоны и начните эфир. При событии должен появиться алерт с камерой, паттерном и таймингами. После накопления данных клип появится в «Видеопроисшествиях»: до 3 секунд до тревоги и 4 после. В самом начале эфира предыстория может быть короче.

Остановите эфир; убедитесь, что клипы доступны, а камера освобождена. Проверьте также кнопку отключения камеры после получения кадра, до старта эфира. Для приёмки используйте безопасную запись, без постановки опасной ситуации на производстве.

Проверка ролей и разделения

Оператор видит эфиры и разбирает события, но не изменяет правила. Мастер видит только назначенные камеры. Юрист получает доказательства без редактирования событий. Менеджмент - статистику. Сотрудник другого предприятия не должен открыть чужой клип, отчёт или эфир даже по прямому URL.

ПРАКТИЧЕСКАЯ ИНСТРУКЦИЯ / ШАГ 08

Качество, ошибки и итоговый пакет

Не менять эталонные метрики при повторном запуске

Сохранённый отчёт docs/quality_report.json: URFD, 20 целых test-последовательностей, event precision/recall/F1 = 0,90; TP=9, FP=1, FN=1. Это локальная оценка RGB-падений, не качество всех промышленных сценариев и не LinQ. scripts/evaluate_fall.py --split test защищён от повторной перезаписи: не удаляйте отчёт ради запуска.

Для переноса сохраните те же тестовые видео, пороги и правила. Сопоставьте выходы TPU с эталоном, затем итоговые события с интервалами разметки. После INT8 не подгоняйте пороги на открытом test. Для новой модели или настройки используйте validation и новый независимый финальный набор.

```
python scripts/evaluate_events.py `
  --truth work/linq-check/truth.json `
  --predictions work/linq-check/tpu-events.json `
  --tolerance 0
```

truth.json: список {video, kind, start_s, end_s}. tpu-events.json: список {video, kind, alert_s}, полученный из реального прогона с теми же video-ID. Дубликат уже сопоставленного события считается FP. Отдельно укажите ложные тревоги на час нормального видео и задержку срабатывания. Этот скрипт оценивает события, а не качество bbox/mAP.

Симптом	Что проверить
Jupyter не открывается	VPN Connected, DNS, TCP:80, конфликт другого VPN, логин нужного сервиса.
Компилятор отклоняет ONNX	Версии SDK/opset, неподдержанные операции, статические формы; сохранить точный лог узла.
Выходы расходятся	RGB/BGR, letterbox, NCHW/NHWC, /255, mean/std, INT8 scale/zero-point, классы и NMS.
Прогон failed или остался PyTorch	LINQ_ADAPTER в процессе сервера, импорт модуля, доступность программы/API; никаких тихих fallback.

Пакет, который сдаём

Git commit; версии SDK/API; ONNX и TPU-программы с SHA256; параметры квантования и источники калибровки; рабочий ноутбук без секретов; подтверждение TPU из сервиса; отчёты 1/2 потоков и p50/p95; предсказания и оценка качества; скриншоты интерфейса; примеры клипов; список ограничений.

Основание: приложенные организаторами «Процедура подготовки и инференса сетей на LinQ TPU» и «Инструкция удалённому сервису инференса сетей LinQ TPU» (страница 1 каждого); код проекта и локальные отчёты. Конкретные API и команды сборки необходимо сверить с полученной версией SDK.